

HACKING WEB APPLICATIONS LAB REPORT

BACKGROUND	
Course:	CIS 170 - Hands-On Ethical Hacking and Network Defense
Term:	Spring 2026
Instructor:	Ray Lampano, Jnr.
Section Number:	15480/15481
Student Name:	Donaven Bruce
Student ID:	[Redacted]
Submission Platform:	Canvas
Report Date:	2026-02-10
Report Version:	1.0
Lab Title:	Hacking Web Applications
Module Reference:	Hacking Web Applications
Lab Type:	Live Virtual Machine Lab
Date Completed:	2026-02-10
Time Spent:	Approximately 1 hour
ENVIRONMENT	
Platforms and Systems:	
- Primary environment: Practice Labs / Live Virtual Machine Labs controlled sandbox. - Primary systems available in the topology: PLABDC01 (Windows Server 2019 - Domain Controller), PLABDM01 (Windows Server 2019 - Domain Member), PLABWIN10 (Windows 10 - Domain Member), and PLABKALI01 (Kali 2019.2 - Linux Kali Workstation). - The work centered mainly on PLABKALI01 for password sniffing (Ettercap) and cracking (Medusa), and on PLABWIN10 for testing bWAPP vulnerabilities (broken authentication, session ID in URL, OS command injection, SSI injection, XSS attacks). - Remote access to the virtual machines was provided through the course platform.	
Context and Constraints:	
- The module focused on web application password cracking, broken authentication, session management, OS command injection, server-side includes (SSI) injection, and cross-site scripting (XSS) attacks. - Practice Labs flagged the module as legacy content using deprecated software, so behavior could vary and some steps were best interpreted as guided sandbox demonstrations. - Windows Updates were disabled in the Practice Labs environment, so software behavior reflected a controlled legacy setup rather than a current hardened enterprise baseline. - The module aligned most closely with exam objectives 3.2 (Information Security Attack Detection), 3.3 (Information Security Attack Prevention), and 4.2 (Information Security Programs).	
Authorization and Scope:	
- All documented activity took place only inside approved academic systems and isolated practice-lab sandboxes. - No public IP space, real organizations, or unauthorized targets were involved. - No production infrastructure, third-party systems, or live services were contacted. - The actions documented here were performed for defensive understanding, web application attack simulation, and ethical hacking education within the course environment. - Restricted material, live credentials, and personal data are omitted or redacted from this report.	
Work Objectives:	
- Review web application vulnerabilities, password cracking techniques, broken authentication, session management issues, command injection, SSI injection, and XSS attacks. - Use Ettercap on PLABKALI01 to sniff passwords from unencrypted sessions and Medusa to crack usernames and passwords against the bWAPP application. - Test multiple bWAPP vulnerabilities on PLABWIN10 (broken authentication, session ID in URL, OS command injection, SSI injection, reflected and stored XSS).	
Hacking Web Applications Concepts and Defensive Framing:	
- The lab identified three core phases of web application attacks: credential capture/cracking, authentication/session bypass, and injection-based exploitation (OS command, SSI, XSS). - Attack types reviewed included password sniffing (Ettercap), brute-force cracking (Medusa), broken authentication, session ID manipulation, OS command injection, SSI injection, and both reflected and stored cross-site scripting. - Tools demonstrated both CLI (Ettercap, Medusa) and web-based (bWAPP) approaches, showing how weak input validation and insecure session handling expose applications. - Prevention concepts reviewed included strong password policies, secure session management, input sanitization, and proper output encoding, reinforcing that web application defense requires both technical controls and secure coding practices.	

TECHNICAL ANALYSIS

Password Cracking and Sniffing on PLABKALI01:

- On PLABKALI01, Ettercap was launched with ettercap -T | grep password to sniff unencrypted sessions. Firefox was used to log into http://192.168.0.10/bWAPP with username bee and password bug, successfully capturing the credentials in cleartext in the terminal.
- A custom wordlist (plab.txt) was created in Leafpad containing common passwords and copied to user.txt. Medusa was then used in multiple modes:
 - medusa -h 192.168.0.10 -u bee -P plab.txt -M ftp (successfully cracked password "bug")
 - medusa -h 192.168.0.10 -u bee -P plab.txt -M ssh (successfully cracked password "bug")
 - medusa -h 192.168.0.10 -U user.txt -p bug -M ssh (identified username "bee")
 - medusa -h 192.168.0.10 -U user.txt -P plab.txt -M ftp (full username/password cracking).

Web Application Vulnerability Testing on PLABWIN10:

- On PLABWIN10, Internet Explorer was used to access http://192.168.0.10/bWAPP and logged in as bee/bug. Multiple bWAPP modules were tested:
 - Broken Authentication - Insecure Login Forms: viewed source code to reveal hard-coded credentials (tonystark / I am Iron Man).
 - Session Management - Administrative Portals: manipulated admin=0 to admin=1 in the URL to bypass access controls.
 - Broken Authentication - Logout Management: logged out but used the browser Back button to regain an active session, demonstrating poor session termination.
 - Session Management - Session ID in URL: observed session ID passed in cleartext in the URL.
 - OS Command Injection: injected |hostname, |net user, |whoami, |pwd, and |cat /etc/passwd to execute system commands and extract sensitive information.
 - Server-side Includes (SSI) Injection: injected <!--#echo var="DOCUMENT_NAME" -->, <!--#echo var="DOCUMENT_URI" -->, and <script>alert('Hacked')</script> to execute SSI directives and JavaScript.
 - Cross-site Scripting (XSS) - Reflected (GET) and Reflected (HREF): injected <script>alert('Hacked')</script> and <script>alert(document.cookie)</script> to execute malicious JavaScript and steal session cookies.
 - Cross-site Scripting (XSS) - Stored (Blog): injected persistent scripts that remained in the database and executed on every page load.

KEY COMMANDS, TOOLS, AND ARTIFACTS BY SYSTEM

PLABKALI01 — Password Cracking & Sniffing Tools:

```
ettercap -T | grep password
leafpad plab.txt
cp plab.txt user.txt
ls -l
medusa -h 192.168.0.10 -u bee -P plab.txt -M ftp
medusa -h 192.168.0.10 -u bee -P plab.txt -M ssh
medusa -h 192.168.0.10 -U user.txt -p bug -M ssh
medusa -h 192.168.0.10 -U user.txt -P plab.txt -M ftp
```

PLABWIN10 — bWAPP Vulnerability Testing:

```
http://192.168.0.10/bWAPP (login: bee / bug)
View source (hard-coded credentials)
URL manipulation: admin=0 →admin=1
Back button after logout
OS Command Injection: |hostname, |net user, |whoami, |pwd, |cat /etc/passwd
SSI Injection: <!--#echo var="DOCUMENT_NAME" ->, <script>alert('Hacked')</script>
XSS Reflected (GET/HREF): <script>alert('Hacked')</script>, <script>alert(document.cookie)</script>
XSS Stored (Blog): persistent <script>alert('Hacked')</script>
```

Primary Tools and Artifacts Observed:

- Ettercap captured cleartext credentials (bee/bug), Medusa cracked FTP/SSH passwords using custom wordlist, bWAPP source code revealing hard-coded credentials, URL manipulation bypassing admin portal, OS command injection executing system commands, SSI directives and JavaScript execution, reflected and stored XSS alerts and cookie theft.
- The most important evidence of successful web application compromise in the lab was the captured/sniffed credentials, Medusa-cracked passwords, command injection output (hostname, users, passwd file), and persistent XSS execution showing stored scripts.

DEFENSIVE MAPPING AND SUMMARY

Technique Summary:

- The completed work covered web application password cracking (sniffing and brute-force), broken authentication, session management issues, OS command injection, SSI injection, and both reflected and stored cross-site scripting using Ettercap, Medusa, and bWAPP.
- The recurring theme across the module was that weak input validation, insecure session handling, and poor credential storage turn web applications into high-value attack surfaces.

MITRE ATT&CK Mapping:

- Credential Access: T1555 Credentials from Password Stores / T1110 Brute Force - Medusa and Ettercap demonstrated credential sniffing and brute-force attacks.
- Initial Access: T1190 Exploit Public-Facing Application - OS command injection, SSI, and XSS directly exploited the bWAPP web application.
- Discovery: T1595.002 Vulnerability Scanning / T1190 - session ID in URL and broken authentication revealed insecure session management.
- Impact: T1499 Endpoint Denial of Service / T1528 Steal Application Access Token - XSS and injection attacks enabled session hijacking and data exfiltration.

Detection Opportunities:

- Unusual login attempts, cleartext credential transmission, repeated failed Medusa-style brute-force patterns, and injected scripts (alert boxes, cookie theft) are strong indicators.
- Web application logs showing OS commands, SSI directives, or reflected/stored XSS payloads can trigger alerts.
- WAF rules for command injection patterns, session ID in URLs, and suspicious JavaScript execution provide early detection.

Defensive Controls:

- Enforce HTTPS everywhere, use strong hashing/salting for passwords, and implement proper session management with secure, HttpOnly cookies.
- Implement strict input validation, output encoding, and parameterized queries to prevent OS command injection, SSI, and XSS attacks.
- Deploy Web Application Firewalls (WAF), rate limiting, and account lockout policies to block brute-force and injection attempts.
- Regular code reviews, security testing, and least-privilege principles reduce the attack surface.

Completion Verification:

- Work was completed across password sniffing/cracking, broken authentication testing, session manipulation, command/SSI injection, and XSS attacks.
- Completion was evidenced by captured credentials, Medusa-cracked passwords, successful injection payloads, and persistent XSS execution.
- The actual commands, tool configurations, and attack payloads used in the lab are preserved directly in this report for accurate documentation and reproducibility.

Key Findings:

- 1) Web applications are prime targets because weak authentication, session handling, and input validation can lead to credential theft, session hijacking, or full system compromise.
- 2) Password sniffing (Ettercap) and brute-forcing (Medusa) succeed against unencrypted or weakly protected credentials.
- 3) Injection attacks (OS command, SSI, XSS) exploit the most common web application flaws when input is not sanitized.
- 4) Session IDs in URLs and poor logout management allow easy session hijacking.
- 5) The strongest defensive takeaway was that secure coding practices, input validation, and proper session management are critical to preventing web application attacks.

Issues / Deviations / Notes:

- The lab used deprecated software and a legacy environment, so exact tool behavior and scan times could vary.
- bWAPP is intentionally vulnerable and was used only inside the isolated lab environment.
- Some injection payloads required multiple attempts due to browser behavior.

Reflection:

- This lab tied together credential capture, authentication bypass, injection attacks, and session manipulation into a complete web application attack workflow.
- The strongest takeaway was that web applications are dangerous because they are publicly exposed and often contain multiple layers of vulnerabilities that can be chained together for full compromise.